

Distributed Job Scheduling Cluster

Fault-Tolerant, Load-Balanced Execution Infrastructure

Distributed Systems Project Team

Politecnico di Milano
090950 – Distributed Systems (A.Y. 2025–2026)
Professors: G. Cugola, A. Margara

August 22, 2026

Core Project Objectives

Project Requirements & Goals

- **Job Submission & Polling:** Clients submit compute tasks to any node, obtain a unique `job_id`, and asynchronously poll for results.
- **Equal Load Sharing:** Executors dynamically balance execution so active load satisfies $|L_i - L_j| \leq 1$.
- **Crash-Recovery Model:** Fail-stop and crash-resume (no Byzantine behavior); nodes rejoin seamlessly.

System Design & Technologies

- **Hybrid Communication Stack:**
 - ▶ **Java RMI:** Point-to-point RPCs (tasks, results & votes).
 - ▶ **UDP Multicast (230.0.0.0:4446):** Decentralized heartbeat gossip and node discovery.
- **Stable Storage:** Disk-based WAL to survive crashes.
- **Code Mobility:** Generic computation via `SerializableSupplier<T>`.

System Architecture Overview

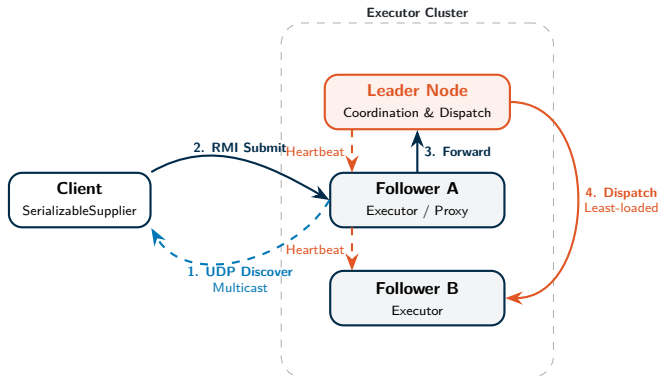
Layered Distributed Design

Client Layer

- Discovers live nodes via **UDP Multicast**.
- Submits **serializable tasks** through RMI.
- Polls task execution status.

Executor Cluster

- **Leader**: coordinates requests and selects the least-loaded executor.
- **Followers**: proxy requests, execute jobs and exchange heartbeats.



Raft-Inspired Consensus Protocol

Consensus Mechanism (ClusterManager)

- **States:** FOLLOWER $\rightarrow$ CANDIDATE $\rightarrow$ LEADER.
- **Monotonic Epochs (Terms):** Strict epoch counter; stale nodes step down if $\text{term}_{\text{incoming}} > \text{term}_{\text{local}}$.
- **UDP Multicast Heartbeats (3.0s):** Periodic beacon broadcasting node metadata (id, ip, port, load, term, isLeader).
- **Randomized Election Timeout:** Nodes wait **5.0–10.0 s** without heartbeat before running for leader (avoids split votes).

Strict Quorum Rule & Configuration

Strict Quorum Rule & Vote Granting

- **Vote Request:** Candidate requests votes via RMI `requestVote(term, candidateId)`.
- **Vote Granting Criteria:** A node votes true if:
 - ▶ $\text{term}_{\text{candidate}} \geq \text{term}_{\text{local}}$.
 - ▶ The node has not already voted for another candidate in the current term (ensured by the Voter WAL).

- **Quorum requirement:**

$$Q = \left\lfloor \frac{\text{expectedClusterSize}}{2} \right\rfloor + 1$$

- Strict majority intersection guarantees at most one leader per term.

Parameter	Configured Value
Heartbeat Interval	3.0 s (UDPMulticastSender)
Election Timeout	[5.0 s, 10.0 s] (Uniform Random)
Node Eviction Timeout	15.0 s (UDP_TIMEOUT)

Job Execution Model & Code Mobility

Code Mobility via `SerializableSupplier<T>`

- Cluster acts as a Generic Compute Engine (FaaS / Serverless model).
- Client serializes arbitrary Java lambda functions inside `Job<T>` without modifying server classes.

Least-Loaded Node Dispatching

- Leader selects executor with lowest load:

$$E_{\text{target}} = \arg \min_{n \in \text{AliveNodes}} (n.\text{getActiveJobs}())$$

- Load balanced across fixed thread pools (4 threads/node).

End-to-End Execution Pipeline:

- ➊ **Ingress:** Client contacts any node $\rightarrow$ transparently routed to Leader's RMI stub.
- ➋ **Dispatch:** Leader assigns job to least-loaded node E_{target} .
- ➌ **WAL Persistence:** E_{target} writes `<jobId>.job` to disk before thread execution.
- ➍ **Execution:** Worker thread computes lambda result.
- ➎ **Result Logging:** Output written to `<jobId>.result` for polling.
- ➏ **Result Retrieval:** Leader scatter-gathers via `getLocalJobResult(jobId)`.

Stable Storage: Dual Write-Ahead Log (WAL)

1. Job Compute WAL (`wal_<nodeId>/`)

- **Disk Records:**

- ▶ `<jobId>.job`: Serialized task input.
- ▶ `<jobId>.result`: Serialized output.

- **Crash-Recovery Sequence on Boot:**

- ① Load all `*.result` files into memory cache.
- ② Scan `*.job` files: unfulfilled tasks are automatically re-executed on the thread pool.

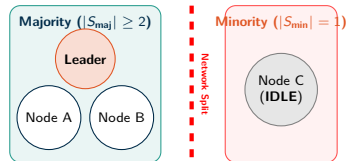
2. Raft Voter WAL (`<nodeId>_raft.properties`)

- **Double-Vote Race Condition:** Node votes for candidate A, crashes, restarts in same term, and votes for candidate B → **split-brain!**
- **Mitigation:** Synchronously persist (`currentTerm`, `votedFor`) to disk before granting any vote (Restored via `loadRaftState()`).

Network Partitions & Split-Brain Mitigation

The expectedClusterSize Invariant

- In case of partition $\{S_{maj}, S_{min}\}$:
- Fixed expectedClusterSize (N) ensures only S_{maj} can reach majority quorum ($\lfloor N/2 \rfloor + 1$).
- **Majority Partition** ($|S_{maj}| \geq Q$): Elects leader, processes jobs, preserves equal load invariant.
- **Minority Partition** ($|S_{min}| < Q$): Fails election and remains in **IDLE** / **Stalled** state.
- Prevents concurrent rogue leaders and state divergence.



Key Architectural Trade-Offs

Design Dimension	Our Solution	Alternative & Trade-Off
Partition Handling	Minority stays IDLE (via <code>expectedClusterSize</code>). Prioritizes consistency and protects equal-load invariant.	<i>Dynamic Re-clustering</i> : Minority elects leader; causes split-brain, duplicate execution, and broken load metrics.
WAL Persistence	Local per-node WAL : Low latency, zero network overhead; suitable for stateless computational jobs.	<i>Replicated Global WAL</i> : High availability during node crashes, but requires heavy distributed consensus per job log.
Client Discovery & Routing	Decentralized Multicast Discovery + RMI Proxying : Client discovers any live node via UDP; Followers transparently route to Leader.	<i>Dedicated Gateway Proxy</i> : Centralizes ingress, but introduces a single point of failure (SPOF) and an extra routing hop.

Conclusions & Key Achievements

Project Summary

- **Hybrid Communication:** Fast RMI point-to-point RPCs combined with UDP Multicast decentralized discovery.
- **Robust Consensus:** Raft terms, random timeouts (5–10s), and strict quorums prevent split-brain.
- **Dual WAL Resilience:** Job WAL protects compute tasks; Voter WAL guarantees single-vote election invariant.
- **Decentralized Architecture:** Direct client multicast discovery and follower transparent forwarding eliminate any single point of failure (SPOF).
- **Code Mobility:** Generic computation via `SerializableSupplier<T>`.

Thank you!

